

CSC111 Project 2 Report: Celebrity Bot

Enaya Amir, Jason Li, Rana Nagash, Zain Mahmoud

March 28, 2025

1 Introduction

Problem: Living in North America, elections and misinformation are a key issue for civic life. In the United States 2024 election, it was reported that an ad campaign powered by large language models (LLMs) was deployed on social media site X to reply to users and spread pro-Republican rhetoric in swing states like Ohio and Pennsylvania (Linville & Warren, 2024). This propaganda strategy has potentially far-reaching consequences for the policies being enacted in those states and the results of local elections. Some of these bots use a strategy that is a variant of voice-cloning, by which an AI model can replicate human speech, either in spoken or written form (Federal Trade Commission's Office of Technology and Division of Marketing Practices, 2023). In our context, we are mainly concerned with the latter: how LLMs can be used to mimic the speech style of celebrity figures to manipulate public opinion.

We chose to focus specifically on replicating the speaking style of influential public figures (including Donald Trump, Martin Luther King, Dr Seuss and Barack Obama) who have a distinct style of speech. Our model can be used to mimic the speaking style of anyone, as long as their corpus is used as the dataset for the model. We show this for example with US president Donald Trump, whose speech style tends to be intuitively recognizable, and arguably most harmful (Niezen, 2025). We were interested in how speech style is replicated to understand the technical background of this wider social issue. As such, we are focusing our research on how these LLMs work as probabilistic decision models.

Our research question is as follows: **How can syntactical speaking style be mimicked by a computer, using graphs and probabilistic pattern matching?**

Goal: We aim to represent the speaking style of a person with a graph of conditional probabilities of that person saying another word given that they have said the previous word. We understand that this is an n-gram language model with a sequence of n words from the training corpus of speeches.

2 Datasets

We modelled our chatbot on speeches spoken by public figures. We obtained the speeches from various GitHub repositories and then cleaned the datasets to suit our purposes. All our datasets are .txt files.

- Various speeches from Donald Trump's 2016 presidential campaign: <https://github.com/ryanmcdermott/trump-speeches/blob/master/speeches.txt>
- Barack Obama's 2009 inaugural address: https://github.com/JuanmaMN/Wordcloud/blob/master/Greatest%20Speeches%20in%20History/Obama_Inaugural_Address.txt
- Martin Luther King Jr's "I have a dream" speech: https://github.com/JuanmaMN/Wordcloud/blob/master/Greatest%20Speeches%20in%20History/I_have_a_dream.txt
- Various Dr Seuss stories: <https://github.com/robertsdionne/rwet/blob/master/hw2/drseuss.txt>

3 Computational Overview

Firstly, we represented each word/n-gram as an instance of our vertex class which has 3 instance attributes: word, which is a tuple of the words, neighbours which is a dictionary mapping the vertices adjacent to this vertex to the weight between them, and the vertex kind which is either a single word or an n-gram. We distinguished between

single words and n-grams (with $n \geq 2$) because there are some words that do not map well to other words alone. For example, the word "the" could map to many other words, but there are significantly fewer words that follow the trigram "the big bad".

The neighbours attribute stores the raw count of how often one vertex (word or n-gram) transitions to another. This is then normalized to probabilities using the `get_neighbours_and_probabilities()` function that uses a helper function to sum the weights and divide the weights by the total.

Moreover, we put together the vertices in the graph class which contains a dictionary of vertices with the keys being the word/n-gram of each vertex and stores the size of the n-grams. The graph also stores a list of the n-grams and words that are currently being used when generating the sentence. We are also storing a list version of the words and n-grams that are currently being used.

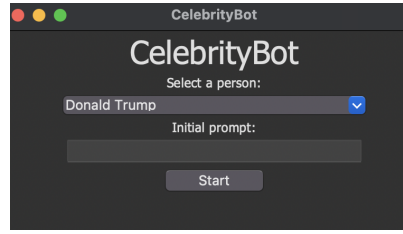
Our graph is unidirectional so this means that it is directed in the sense that v_1 might be a neighbour of v_2 but v_2 might not necessarily be a neighbour of v_1 , depending on the sentence.

The graph is constructed using the `_parse_text` method which parses the text from the corpus and creates the tokens using the NLTK library. We then loop through the tokens and add every n tokens as a vertex to the graph. The neighbours of each vertex are the tokens that follow it and so we add an edge between these vertices using the `add_edge` method which takes in two items and adds an edge between them if they are already in the graph, raises an error otherwise. To add an edge, we use the `_Vertex` method `add_neighbour` which takes a vertex and checks if it is already a neighbour of the current vertex. Since neighbours are stored in a dictionary that maps the adjacent vertices to their weight with `self` (which is the number of times the adjacent vertex appears after `self`), we increment that weight of the vertex we are adding if it is already a neighbour or we set the count to 1 if it is the first time the word we are adding appears after `self`.

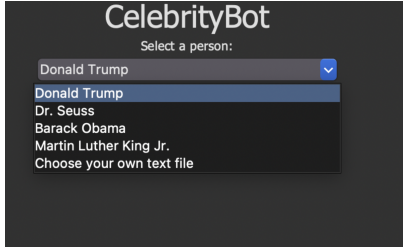
Once the graph has been constructed, we use the `predict_next_word` method which takes in a starting word or n-gram as a parameter. It then searches for that starting word/n-gram in the graph's vertices. If the word is not in the text file, we choose the closest word to it using the `diffib` methods from the `diffib` python library. Then, we choose the next word based on the conditional probability of the adjacent vertices (we calculate the conditional probability of obtaining vertex1 after vertex2 by dividing the weight of the edge between vertex1 and vertex2 / the total weight of all the vertices adjacent to vertex2) using the NumPy random choice method. We then call the `predict_next_word` method recursively on the chosen word until the "stop" button is pressed.

Furthermore, this is a Markov chain chatbot that uses conditional probabilities to predict the next word. The user is prompted to input a word and the chatbot will output text similar in speaking style to the celebrity chosen. To understand how well the model is performing, we used the BLEU (Bilingual Evaluation Understudy). BLEU's output is a number between 0 and 1 and values closer to 1 represent a better match between our model and the reference texts (the corpus). We evaluated the chatbot's performance using the BLEU score in the `bleuscore.py` file. This was a test between some sentences from the corpus of Donald Trump as a reference and the output our chatbot gave as the hypothesis. Both the reference texts and the hypothesis started with the word "president". This gave us a BLEU score of 0.64, which means that our chatbot is moderately similar to the corpus of Donald Trump.

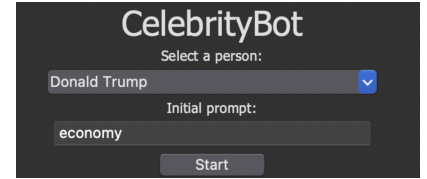
We also created a graphical user interface for the chatbot in the `main.py` file using the TKinter library. We used this library to draw the menu and add the buttons. When run, the user is prompted with a menu where they can choose the celebrity they would like to mimic from the given list (or upload their own text file) and then input their starting word as seen in **Figure 1**. Then, the program displays the sentences until the user presses the stop button.



(a) Initial menu



(b) Choosing celebrity



(c) Input starting word

Figure 1: Graphical User Interface

4 Graph Visualization

To visualize the conditional probabilities from one word to the next, we constructed a unidirectional graph using `networkx` and `plotly` as seen in the `visualize_graph_plotly()` method. This method takes in a `Graph` object (created by the parsing of text) and an integer `n` representing the maximum number of vertices the visualization should display, chosen at random. The visualization starts taking too long to load after 200 vertices.

We chose to visualize word frequencies (edge weight) by the distance of each edge in the graph. The `DiGraph()` function from `networkx` was used to create a directed graph with positioning given by `nx.spring_layout()`. To plot the vertices and edges, we used `plotly.graph_objs.Scatter()` and `add_trace()` functions, using the hover feature to display the words associated with each vertex. The final output layout was determined by `figure.update_layout()`. The following figures are some example graphs with different parameters and people.

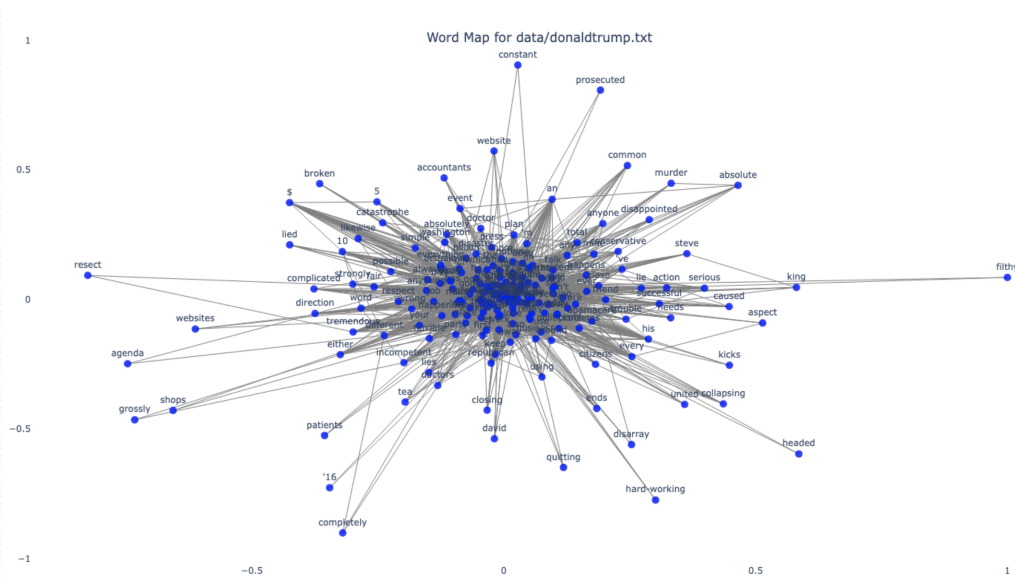


Figure 2: Trump, 200 vertices, full graph

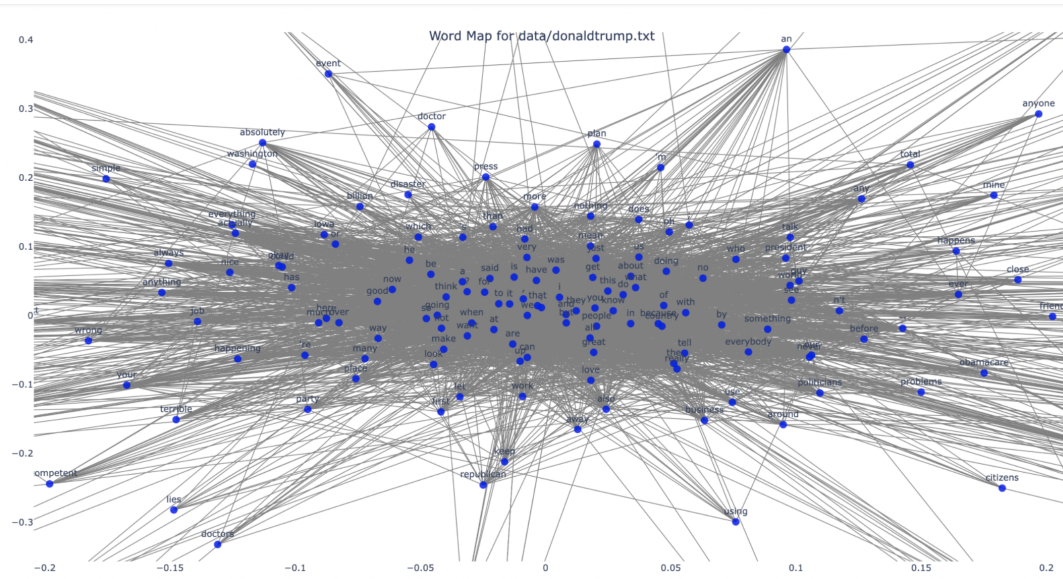


Figure 3: Trump, 200 vertices, zoomed in

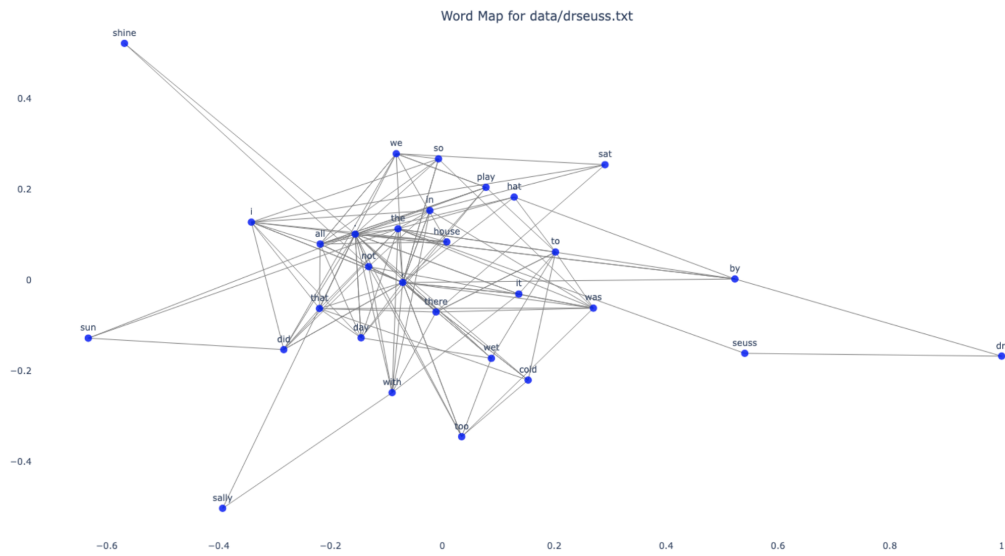


Figure 4: Dr. Seuss, 30 vertices

5 Running the Project - Instructions

Setup:

- All the Python files should be stored in the root directory of the project.
- The datasets will be uploaded to MarkUs as a zip file called "data.zip". The text files should be stored in a folder named "data", and the folder should be stored where the Python files are stored. See the image below for reference of the file structure.
- If you're running the file on Mac, ensure to download the SSL (navigate to **Applications/Python 3.x** (where x is your version of Python) and double-click on **Install Certificates.command**. This should install the necessary certificates for Python to verify SSL connections.)

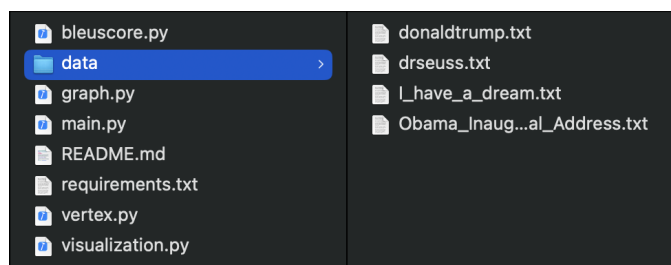


Figure 5: File structure

- Install all the dependencies in the "requirements.txt" file (e.g. `pip install -r requirements.txt` in your terminal). You may also need to run `python -m nltk.downloader punkt.tab`

Using the program:

1. To run the program, simply run the `main.py` file (e.g. `python main.py` in your terminal) and the interface will open in a window. The interface will contain everything to interact with the program.
2. To begin, either select one of the included people under the dropdown, or select the option to use your own text file of data in the dropdown. If you choose the latter option, a file dialog will appear after you click the "Start" button.
3. Type a prompt to start the text generation. We suggest typing something that sounds like the person you have chosen, for best accuracy, however it will work with any input.
4. Click the start button. The text should start generating indefinitely. Click the stop button to stop. You can copy the output by using the copy output button.
5. You can use the visualize button to visualize the graph. This uses your selected person/text file, but not your prompt. You will be prompted to select the number of vertices to show in the visualization. This is because for large files, it will run poorly (or not at all) if the entire graph is displayed. It will display a subset of the graph.

6 Changes from Proposal

Initially, we were only aiming to replicate Donald Trump's speech. However, we extended our project to include other celebrities and added the option to upload a custom dataset. We also added an evaluation metric using the BLEU score as explained above.

7 Discussion

Our chatbot helps explain and show how syntactical speaking style can be mimicked by a probabilistic language model. By using the evaluation metric, we explored how closely our chatbot can mimic the speaking style of an influential figure like Donald Trump. We saw that it was moderately similar due to the BLEU score of 0.64. We realize that getting an output of 1 is not feasible since our chatbot cannot be an exact replica to a human's speech. The BLEU score can improve with better machine learning/reinforcement learning techniques.

It is clear that chatbots can be used to mimic the speaking style of influential figures, hence worsening the broader social issue of misinformation and propaganda being spread. However, they can never be an exact replica of human speech. This difference could remain unnoticeable to the wider population.

While working on this project, we inevitably encountered some obstacles. Firstly, we were unsure of how to implement the BLEU metric so that it would best represent the performance of our chatbot. When performing the evaluation on a sample for Donald Trump's speech, we did not know what kind of reference text we should be comparing the output text to; the entire corpus or a specific sentence that generally represents Donald Trump's

speaking style. Finally, we decided to reference it to sentences in the corpus that follow the same first word that the user inputs.

We also encountered issues and limitations with our visualization due to our large dataset. Since some text files contained thousands of words and thus thousands of vertices in their graphs, the visualization often crashed or took several minutes to produce a graph, and when it did produce a graph, it would be extremely dense. To fix this, we introduced a limit to the number of vertices that would be displayed. This limit is the parameter `n` passed into `visualize_graph_plotly()`. We considered visualizing word frequency by vertex size, however, since some words appeared far more frequently than others, our visualization became highly unbalanced and hard to read, so this feature was dropped. We also considered modifying the hover feature to display a list of each vertex’s neighbours. However, once again, since the graph could be very dense, these lists could become clunky and hard to read, so they were removed.

Our program’s results are represented to the user interactively through a chatbot. After the user enters the prompt, the sentences are displayed to the user as they are generated (similar to other LLMs like ChatGPT) until the user presses the “stop” button. We decided that this was better than stopping the text generation with certain end-of-sentence punctuation as the chatbot would often end abruptly. The graph is also displayed to the user using `plotly` which loads the graph visualization in a web browser. The visualization button can be accessed through the GUI.

Some next steps for further exploration include using various evaluation metrics like p-values and/or bootstrapping to evaluate the chatbot’s performance relative to the given corpus. This can also be used as a reward/feedback system where the chatbot predicts sentences that yield higher scores (which are generally closer to the true sentence). Also, we can use larger corpora and use an 80-20 train-test data split to test the chatbot’s output on the given corpora. This will ensure that the output is similar to the person’s speech.

To conclude, our chatbot addresses our primary concern of how LLMs can be used to mimic the speech style of celebrity figures. It explores the depths of how syntactical speaking style is mimicked by a computer and shows probabilistic pattern matching through an n-gram language model. Chatbots like these can be used for a variety of different purposes; whether it is to manipulate public opinion or create new and engaging content for an audience. Overall, mimicking the syntactical style of someone *is* possible, yet it is crucial to consider the purpose of doing so to decide if it is beneficial.

References

- Linville, D., & Warren, P. (2024). *Digital yard signs: Analysis of an AI bot political influence campaign on X*. Retrieved from https://open.clemson.edu/cgi/viewcontent.cgi?article=1006&context=mfh_reports
- Federal Trade Commission. (2023). *Preventing the harms of AI-enabled voice cloning*. Office of Technology and Division of Marketing Practices. Retrieved from <https://www.ftc.gov/policy/advocacy-research/tech-at-ftc/2023/11/preventing-harms-ai-enabled-voice-cloning>
- Niezen, R. (2025). ‘Aliens’ and ‘animals’: Language of hate used by Trump and others can be part of a violent design. *NBC News*. Retrieved from <https://www.nbcnews.com/tech/internet/republican-bot-campaign-trump-x-twitter-elon-musk-fake-accounts-rcna173692>